Dart Programming

Session 1

Introduction to dart

What is Dart?

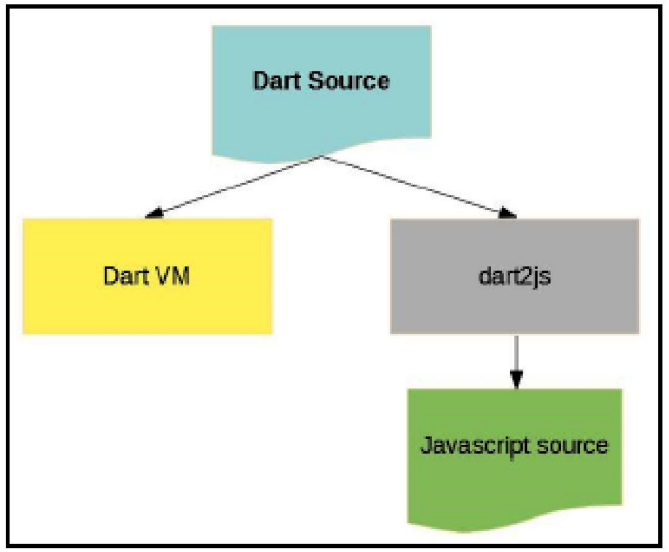
- Dart is an object-oriented programming (OOP) language, has a class-based style, and uses a C-style syntax.
- If you are familiar with the C#, C++, Swift, Kotlin, and Java/JavaScript languages, you will be able to start developing in Dart quickly.
- Syntactically, Dart bears a strong resemblance to Java, C, and JavaScript. It is a dynamic object-oriented language with closure and lexical scope. The Dart language was released in 2011 but came into popularity after 2015 with Dart 2.0.
- But don’t worry—even if you are not familiar with these other languages, Dart is a straightforward language to learn, and you can get started relatively quickly.

Why Use Dart

- Fast & Smooth: Dart compiles to native code for speedy performance, ideal for mobile apps.
- Easy to Learn: Similar to familiar languages like Java or Javascript, making it approachable for new developers.
- Flutter Power: Dart is the heart of Flutter, a popular framework for building beautiful and functional mobile apps.
- One Code, Many Places: Develop for mobile, web, and even desktop with a single codebase (primarily with Flutter).

How Dart works

- To understand where the language's flexibility came from, we need to know how we can run Dart code. This is done in two ways:
  - Dart Virtual Machines (VMs)
  - JavaScript compilations
- Dart code can be run in a Dart-capable environment. A Dart-capable environment provides essential features to an app, such as the following:
  - Runtime systems
  - Dart core libraries
  - Garbage collectors



- The execution of Dart code operates in two modes—Just-In-Time (JIT) compilation or Ahead-Of-Time (AOT) compilation

How Dart works

- A JIT compilation is where the source code is loaded and compiled to native machine code by the Dart VM on the fly. It is used to run code in the command-line interface or when you are developing a mobile app in order to use features such as debugging and hot reloading.
- An AOT compilation is where the Dart VM and your code are precompiled and the VM works more like a Dart runtime system, providing a garbage collector and various native methods from the Dart software development kit (SDK) to the application.
- Flutter's hot reload is one of its most famous features and shows the promised productivity in action.
- It relies on JIT compilation to make live Dart code swaps while running the app, so we can change our application code and see the result almost in real time.
- With IDE plugins, this becomes even faster as, after saving a change, the plugin dispatches the reload and the result is seen quickly.

Installing Dart

- Step 1: Download Dart SDK
  - Go to Dart SDK archive page.
  - The URL is https://dart.dev/tools/sdk/archive.
- Step 2: Extract zip file
- Step 3: Add Dart Path to PATH Environment Variable
  - Open Environment Variables. Under System variables, click on Path and click Edit button.
  - Edit environment variable window appears. Click on New and paste the dart sdk bin path
- Step 4: Run Dart
  - Run the command dart.

Installing Dart

- You can install the Dart SDK using Chocolatey.
  - https://chocolatey.org/install#individual
- Important: These commands require administrator rights. Here's one way to open a Command Prompt window that has admin rights:
- To install the Dart SDK:
  - choco install dart-sdk
- To upgrade the Dart SDK:
  - choco upgrade dart-sdk
- If you can't use the Dart SDK executables, add the SDK location to your PATH:
  - In the Windows search box, type env.
  - Click Edit the system environment variables.
  - Click Environment Variables….
  - In the user variable section, select Path and click Edit….
  - Click New, and enter the path to the dart-sdk directory.
  - In each window that you just opened, click Apply or OK to dismiss it and apply the path change.

Hello World !

Hello World

Every app requires the top-level `main()` function, where execution starts. Functions that don't explicitly return a value have the `void` return type. To display text on the console, you can use the top-level `print()` function:

```dart
void main() {
  print('Hello, World!');
}
```

Hello World Explanation

- main(): it is the symbol of main function that means the data entered in it is directly executed by compiler.
- print(“Hello World!”) : the role of print function is quite simple it just prints the data during the compilation of a program.

Comments

- In every programming language comments play an important role for a better understanding of the code in the future or by any other programmer.
- Comments are a set of statements that are not meant to be executed by the compiler. They provide proper documentation of the code.
- Types of Dart Comments:
  - Dart Single line Comment.
  - Dart Multiline Comment.
  - Dart Documentation Comment.

Single Line comment

- 1. Dart Single line Comment: Dart single line comment is used to comment a line until line break occurs. It is done using a double forward-slash (//).

```
// This is a single line comment.
```

- main() {
- double area = 3.14 * 4 * 4;
- // It prints the area
- // of a circle of radius = 4
- print(area);

}

Dart Multi-Line Comment

- Dart Multiline comment is used to comment out a whole section of code.
- It uses ‘/*’ and ‘*/’ to start and end a multi-line comment respectively.

```
/*
These are
multiple line
of comments
*/
```

Dart Documentation Comment

- Dart Documentation Comments are a special type of comment used to provide references to packages, software, or projects.
- Dart supports two types of documentation comments
  - “///”(C# Style)
  - “/**…..*/”(JavaDoc Style).
- It is preferred to use “///” for doc comments as many times * is used to mark list items in a bulleted list which makes it difficult to read the comments.
- Doc comments are recommended for writing public APIs.

```
/// This is

/// a documentation

/// comment
```

Variables

Variables

- A variable is used to represent a value and to hold onto it for future use.
- (=) is used in Dart for assignment (assigning values to variables).
  - var x = 5;
- In Dart, this code assigns the value of 5 to the variable x. Similarly, the next example assigns the string literal "antelope" to the variables animalWord.
  - var animalWord = "antelope";
- Variables may also have a *type*.
  - String coolWord = "antelope";
- Types are optional in Dart. However, they are useful when we want to ensure the safety of some of our variables by enforcing a specific type. Types are also helpful when debugging.
- Even in type-safe Dart code, you can declare most variables without explicitly specifying their type using var. Thanks to type inference, these variables' types are determined by their initial values:
- Technically, all variables in Dart are references to objects.

Conditions to Write Variable Name

- Conditions to write variable names or identifiers are as follows:
  - Variable names or identifiers can’t be the keyword.
  - Variable names or identifiers can contain alphabets and numbers.
  - Variable names or identifiers can’t contain spaces and special characters, except the underscore(_) and the dollar($) sign.
  - Variable names or identifiers can’t begin with a number.
- Note: Dart supports type-checking, it means that it checks whether the data type and the data that variable holds are specific to that data or not.

```
var name = 'Voyager I';
var year = 1977;
var antennaDiameter = 3.7;
var flybyObjects = ['Jupiter', 'Saturn', 'Uranus', 'Neptune'];
var image = {
  'tags': ['saturn'],
  'url': '//path/to/saturn.jpg'
};
```

Variables

- There are six special types in Dart that are fundamental to the language and can be described with literals.
- These are integers, floating-point numbers, strings, booleans, lists, and maps.

Table 3-1. *Fundamental Dart Types That Can Be Described with Literals*

Type	Description	Example Literals
`int`	Integers	`5, -20, 0`
`double`	Floating-Point Numbers	`3.14159, -3.2, 0.00`
`String`	Strings	`"hello", "g", "To be or not to be?"`
`bool`	Booleans	`true, false`
`List`	Lists (Chapter 6)	`[1,2,3], ["hi", "bye"]`
`Map`	Maps (Chapter 6)	`{"x": 5, "y":2}`

Dynamic type

- This is a special variable initialised with keyword dynamic.
- The variable declared with this data type can store implicitly any value during running the program.
- It is quite similar to var datatype in Dart, but the difference between them is the moment you assign the data to variable with var keyword it is replaced with the appropriate data type.
- // Assigning value to geek variable
- dynamic message = “Hello Dart";
- // Printing variable geek
- print(message);
- // Reassigning the data to variable and printing it
- message = 3.14157;
- print(message);

Final & Constants

- If you never intend to change a variable, use final or const, either instead of var or in addition to a type.
- These keywords are used to define constant variable in Dart i.e. once a variable is defined using these keyword then its value can’t be changed in the entire code. These keyword can be used with or without data type name.
- A final variable can be set only once; a const variable is a compile-time constant. (Const variables are implicitly final.)
- Instance variables can be final but not const.

Final

- A final variable can only be set once and it is initialized when accessed.
- // Without datatype
- final variable_name
- // With datatype
- final data_type variable_name

Final vs Const

- Use const for variables that you want to be compile-time constants
- Where you declare the variable, set the value to a compile-time constant such as a number or string literal, a const variable, or the result of an arithmetic operation on constant numbers:
- Although a final object cannot be modified, its fields can be changed. In comparison, a const object and its fields cannot be changed: they're immutable.

```dart
const bar = 1000000; // Unit of pressure (dynes/cm2)
const double atm = 1.01325 * bar; // Standard atmosphere
```

Null Safety

Null Safety

- The Dart language enforces sound null safety.
- Null safety prevents an error that results from unintentional access of variables set to null.
- The error is called a null dereference error.
- A null dereference error occurs when you access a property or call a method on an expression that evaluates to null.
- With null safety, the Dart compiler detects these potential errors at compile time.
- By Default, every variable type in dart is non-nullable.
- If you want the variable type to be nullable, you should explicitly add (?) after the type
- For example, say you want to find the absolute value of an int variable i. If i is null, calling i.abs() causes a null dereference error. In other languages, trying this could lead to a runtime error, but Dart's compiler prohibits these actions. Therefore, Dart apps can't cause runtime errors.

Null Safety

- When you specify a type for a variable, parameter, or another relevant component, you can control whether the type allows null. To enable nullability, you add a ? to the end of the type declaration.

```
String? name  // Nullable type. Can be `null` or string.

String name   // Non-nullable type. Cannot be `null` but can be string.
```

- You must initialize variables before using them.
- Nullable variables default to null, so they are initialized by default.
- Dart doesn't set initial values to non-nullable types. It forces you to set an initial value. Dart doesn't allow you to observe an uninitialized variable.

Null Safety

- You can't access properties or call methods on an expression with a nullable type. The same exception applies where it's a property or method that null supports like hashCode or toString().
- Uninitialized variables that have a nullable type have an initial value of null. Even variables with numeric types are initially null, because numbers—like everything else in Dart—are objects

```dart
int? lineCount;
assert(lineCount == null);
```

- With null safety, you must initialize the values of non-nullable variables before you use them:

```
int lineCount = 0;
```

Null Safety

- You don't have to initialize a local variable where it's declared, but you do need to assign it a value before it's used. For example, the following code is valid because Dart can detect that lineCount is non-null by the time it's passed to print():

```dart
int lineCount;

if (weLikeToCount) {
  lineCount = countLines();
} else {
  lineCount = 0;
}

print(lineCount);
```

Null Awareness Operators

Dart offers some handy operators for dealing with values that might be null. One is the `??=` assignment operator, which assigns a value to a variable only if that variable is currently null:

```dart
int? a; // = null
a ??= 3;
print(a); // <-- Prints 3.

a ??= 5;
print(a); // <-- Still prints 3.
```

Null Awareness Operators

Another null-aware operator is `??`, which returns the expression on its left unless that expression's value is null, in which case it evaluates and returns the expression on its right:

```dart
print(1 ?? 3); // <-- Prints 1.
print(null ?? 12); // <-- Prints 12.
```

Conditional property access

To guard access to a property or method of an object that might be null, put a question mark (`?`) before the dot (`.`):

```dart
myObject?.someProperty
```

Late Initializer

- The late modifier has two use cases:
  - Declaring a non-nullable variable that's initialized after its declaration.
  - Lazily initializing a variable.
- Often Dart's control flow analysis can detect when a non-nullable variable is set to a non-null value before it's used, but sometimes analysis fails. Two common cases are top-level variables and instance variables: Dart often can't determine whether they're set, so it doesn't try.
- If you're sure that a variable is set before it's used, but Dart disagrees, you can fix the error by marking the variable as late:

```dart
late String description;

void main() {
  description = 'Feijoada!';
  print(description);
}
```

Late Initializer

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

> ⚠ Notice
>
> If you fail to initialize a `late` variable, a runtime error occurs when the variable is used.

- When you mark a variable as late but initialize it at its declaration, then the initializer runs the first time the variable is used. This lazy initialization is handy in a couple of cases:
  - The variable might not be needed, and initializing it is costly.
  - You're initializing an instance variable, and its initializer needs access to this.
- In the following example, if the temperature variable is never used, then the expensive readThermometer() function is never called:

```dart
// This is the program's only call to readThermometer().
late String temperature = readThermometer(); // Lazily initialized.
```